

MTE 204 – Numerical Methods

Tutorial #1 Solutions

Question 1:

The volume $\bar{V}$ of liquid in a spherical tank of radius R is related to the depth of the liquid h by the following equation.

$$\bar{V} = \frac{\pi h^2(3R - h)}{3}$$

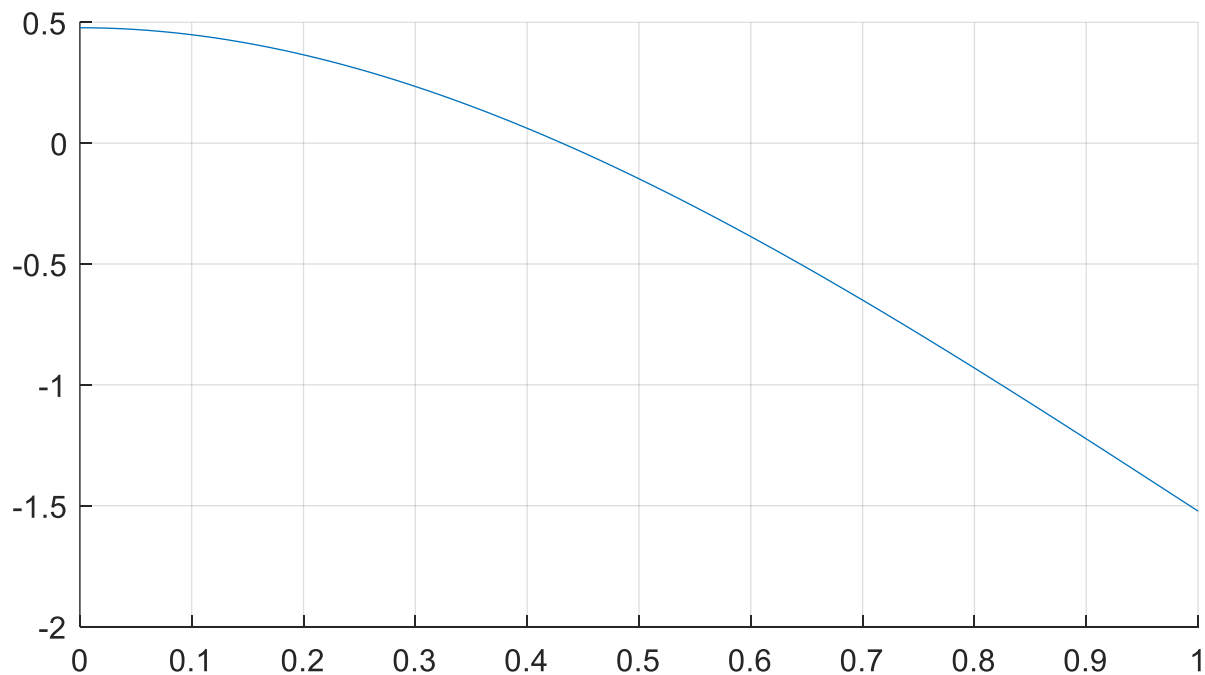
For a spherical tank with a radius of 1 m and containing 0.5 m³ of liquid, find the depth of the liquid in the tank. Use fixed point iteration for this problem.

Solution:

First, let us reorganize our equation for volume such that all terms are on the right hand side. Let us further expand our terms to form a standardized polynomial expression.

$$0 = h^3 - 3Rh^2 + \frac{3}{\pi}\bar{V}$$

Lets plot this function for a range of heights to determine the overall shape, number of roots, and approximate location of those roots. Based on this graph (below), **we can see there is a single root between $h = 0.4$ and $h = 0.5$.**



The MATLAB code used to produce this graph is in the box below. Plotting requires creating an array of **heights h** , then sequentially evaluating the function at each height. In this case, we use a for loop.

```
% Initialize h array to be used for plotting. linspace(a,b,n) creates an
% array of n points between a and b
h = linspace(0,1,100);

% compute the function at all points in x
for i = 1:length(h)
    y(i) = h(i)^3-3*R*h(i)^2+3/pi*V;
end

% Create the plot
figure(); % This forces matlab to create a new figure
hold on; % by default matlab overwrites every plot its makes. This stops
that behaviour
% create a line plot
plot(h,y)
grid on; % enables grid lines on the plot
% Set x plot limits. Matlab sets y-limits automatically to match
xlim([0,1])
```

To perform fixed-point iteration, we must **modify our equation to be in the form $g(x) = x$** , where x is the variable **we are solving for (h in this case)**. **There are two options for doing this:** 1) we can add x to both sides or 2) rearrange the equation to place and isolated x in the equation. Let's try both options.

Recall, the equation we are trying to find roots for is

$$0 = h^3 - 3Rh^2 + \frac{3}{\pi}\bar{V}$$

OptionX 1: Add h to both sides

$$g(x) = x$$

$$h = h^3 - 3Rh^2 + h + \frac{3}{\pi}\bar{V}$$

$$g(h) = h^3 - 3Rh^2 + h + \frac{3}{\pi}\bar{V}$$

Option 2: Rearrange equation to place an isolated h on one side

Let's rearrange our initial equation to place the squared h on the left-hand side

$$3Rh^2 = h^3 + \frac{3}{\pi}\bar{V}$$

Now isolate h on the left-hand side.

$$h = \sqrt{\frac{h^3 + \frac{3}{\pi} \bar{V}}{3R}}$$

$$g(h) = \sqrt{\frac{h^3 + \frac{3}{\pi} \bar{V}}{3R}}$$

We have now created two different $g(h)$ for the same equation. Will both converge to the same answer? Let's find out. Starting with an initial guess of $h_0 = 0.5$, the table below details the first six iteration of the fixed-point method ($h = [0, 0.0101, 0.0202, 0.0303, \dots, 0.9899, 1.0000]$)

ITERATION	OPTION 1: ADD h TO BOTH SIDES	OPTION 2: REARRANGE EQUATION
0	0.5	0.5
1	0.35246	0.44813
2	0.50102	0.43492
3	0.35119	0.43195
4	0.50197	0.43130
5	0.35000	0.43116
6	0.50284	0.43113

For reference, the real root is **0.43112**, which our second option converges to within six iterations. Unfortunately, using the first and easiest option (adding h to both sides) does not produce a convergent solution. Fixed-point iteration is not always guaranteed to converge. **Fortunately, we can check if an equation will converge before we start iterating.**

From the textbook (Box 6.1), we know that fixed-point iteration will converge if $|g'(x)| < 1$. (**This indicates that the error decreases after each iteration.**) Let's take the derivative of the equations used in both options and test the derivative with our initial guess of $h_0 = 0.5$.

Option 1: Adding h to both sides

Original equation

$$g(h) = h^3 - 3Rh^2 + h + \frac{3}{\pi} \bar{V}$$

Taking the derivative yields

$$g'(h) = 3h^2 - 6Rh + 1$$

At $h = 0.5$, $|g'(0.5)| = 1.25$. Because this is greater than 1, we know before we start that this formulation will not converge by fixed-point iteration.

Option 2: Rearranging to isolate h

Original equation

$$g(h) = \sqrt{\frac{h^3 + \frac{3}{\pi} \bar{V}}{3R}}$$

Taking the derivative yields the following (hint: use chain rule!)?

$$g'(h) = \frac{h^2}{2\sqrt{\frac{h^3}{3R} + \frac{\bar{V}}{\pi R}}}$$

At $h = 0.5$, $|g'(0.5)| = 0.1225$. **Because this value is less than 1**, we know this formulation will converge using fixed-point iteration.

While the derivative can be messy, it is always best to check for convergence before you start iterating. There is no hard and fast rule as to if it is better to add x to both sides or rearrange the equation to isolate x . **You must always check for convergence.**

This is one of the reasons the other root finding methods discussed in this course are more commonly used. **They typically don't have the degree of sensitivity to problem formulation.**

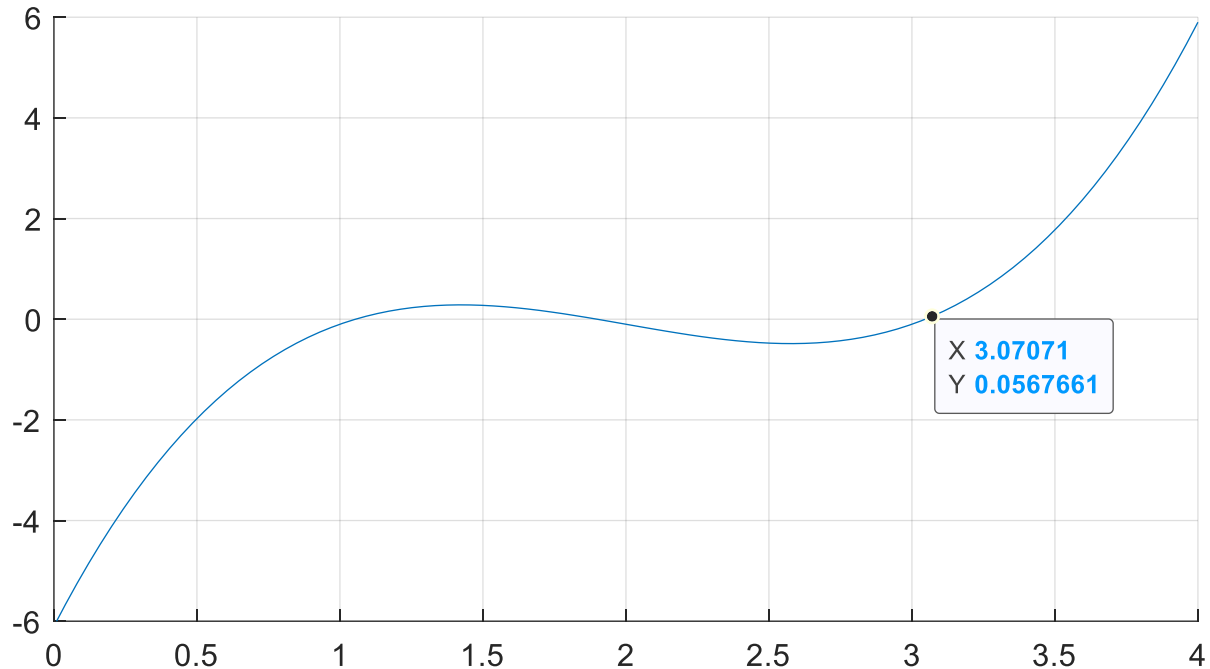
Question 2:

Determine the largest real root of $f(x) = x^3 - 6x^2 + 11x - 6.1$ using the following two methods

- The graphical method
- The Newton-Raphson method. Perform 3 iterations using $x_0 = 3.5$ (initial guess)

Solution:

Part a) Graphical Method



MATLAB code used to generate this plot. Here we make use of MATLAB's element-wise operators to compute all values of y in a single line of code instead of using a for loop.

```
% Initialize x array to be used for plotting. linspace(a,b,n) creates an
% array of n points between a and b
x = linspace(0,4,100);
% compute the function at all points in x. In this problem, we forgo using
% a for loop to make use of Matlab's element-wise operators. Using .* and
% .^ performs multiplication and exponents for every element in the initial
% array. Addition and subtraction do not need a dot.
y = x.^3 - 6.*x.^2 + 11.*x - 6.1;
% Create the plot
figure(); % This forces matlab to create a new figure
hold on; % by default matlab overwrites every plot its makes. This stops
that behaviour
% create a line plot
plot(x,y)
grid on; % enables grid lines on the plot
% Set x and y plot limits. Matlab sets these automatically otherwise
xlim([0,4])
ylim([-6,6])
```

Part b) Newton-Raphson Method

Recall the function is defined as:

$$f(x) = x^3 - 6x^2 + 11x - 6.1$$

Taking the derivative yields:

$$f'(x) = 3x^2 - 12x + 11$$

The Newton-Raphson method is defined as

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Let's see the Matlab

Three iterations of this Newton-Raphson method are detailed in the table below. After three iterations, the solution converges to 3.04668.

ITERATION	x_i	$f(x_i)$	$f'(x_i)$	x_{i+1}
0	3.50000	$1.7750e + 00$	$5.7500e + 00$	3.19130
1	3.19130	$3.9940e - 01$	$3.2576e + 00$	3.06869
2	3.06869	$5.1880e - 02$	$2.4263e + 00$	3.04731
3	3.04731	$1.4560e - 03$	$2.2906e + 00$	3.04668

%% Part b - Newton-Raphson Method

xOld = 3.5; % Set initial guess

iterMax = 3;

% Begin iterations. Use for loop because we are limiting to 3 iterations

for iter = 1:iterMax+1

 % compute function and derivative at x_i

 f = xOld^3 - 6*xOld^2 + 11*xOld - 6.1;

 fPrime = 3*xOld^2 - 12*xOld + 11;

 % Compute x_i+1

 xNew = xOld - f/fPrime;

 % The following code displays the output in a cleanly formatted way

 if iter == 1

```

    fprintf('iter | x_i      | f(x_i)    | f'(x_i)    | x_{i+1} \n')
end

fprintf('%4d | %13e | %13e | %13e | %13e \n', iter-1, xOld, f, fPrime, xNew)

% Replace x_i with x_{i+1}
xOld = xNew;

% At this point you would add code to check convergence.

end

```

Question 3:

The velocity of a falling parachutist is given by $v = \frac{gm}{c} \left(1 - e^{-\left(\frac{c}{m}\right)t} \right)$. Using the bisection method, determine the drag coefficient needed such that an **80 kg (m)** parachutist has a velocity of **35 m/s (v)** after **3.9 s (t)** of freefall. Assume gravity is **9.81 m/s² . (g)** Use an initial bracket of [3,5] and iterate until approximate relative error falls below 5%. Perform an error check by substituting your final answer into the initial equation.

Solution:

The first step is to reorganize the velocity equation to place all terms on the right-hand side.

$$f(c) = \frac{gm}{c} \left(1 - e^{-\left(\frac{c}{m}\right)t} \right) - v$$

Our goal is to find the value of c such that $f(c) = 0$.

Since the problem asks us to use the bisection method, we start with the given bracket [3,5]. The main idea of bisection is that if the function values at two points have opposite signs, then the root lies between those two points.

This solution manual will detail the first two iterations

Iteration 1: The bounds for this iteration are [3,5]

$$x_{mid} = \frac{x_{low} + x_{upp}}{2} = \frac{3 + 5}{2} = 4$$

$$x_{low} = 3, \quad f(x_{low}) = 0.59285$$

$$x_{mid} = 4, \quad f(x_{mid}) = -0.24016$$

$$x_{upp} = 5, \quad f(x_{upp}) = -1.046967$$

$$f(x_{low})f(x_{mid}) = f(3)f(4) = -0.14239$$

Since $f(x_{low})f(x_{mid}) = -0.14239 < 0$, we know the root falls between [3, 4]. So, we replace x_{upp} with x_{mid} . The error at this iteration is $\left| \frac{3-4}{4} \right| = 0.250$. Since this error is greater than 5%, we continue to the next iteration.

Iteration 2: The bounds for this iteration are [3,4]

$$x_{mid} = \frac{x_{low} + x_{upp}}{2} = \frac{3 + 4}{2} = 3.5$$

$$x_{low} = 3, \quad f(x_{low}) = 0.59285$$

$$x_{mid} = 3.5, \quad f(x_{mid}) = 0.173011$$

$$x_{upp} = 4, \quad f(x_{upp}) = -0.24016$$

$$f(x_{low})f(x_{mid}) = f(3)f(3.5) = 0.10257$$

Since $f(x_{low})f(x_{mid}) = 0.10257 > 0$, we know the root falls between $[3.5, 4]$. So, we replace x_{low} with x_{mid} . The error for this iteration is $\left| \frac{3.5-4}{3.5} \right| = 0.1429$. Since this error is greater than 5%, we continue to the next iteration.

So far, we have done the bisection method manually. We calculated the midpoint, evaluated the function, checked the sign, updated the bracket, and computed the approximate relative error.

However, if the tolerance is much smaller, or if we need many more iterations, doing this by hand becomes inefficient. This is exactly where MATLAB becomes useful. MATLAB can repeat the same procedure automatically using a loop.

The table below details the first four iterations of the bracketing method for this problem

ITER	x_{low}	x_{mid}	x_{upp}	$f(x_{low})$	$f(x_{mid})$	$f(x_{upp})$	$f(x_{low})f(x_{mid})$	ERROR
1	3	4	5	0.59285	-0.24016	-1.04697	-0.14238	0.2500
2	3	3.5	4	0.59285	0.17301	-0.24016	0.10257	0.1429
3	3.5	3.75	4	0.17301	-0.03440	-0.24016	-0.00595	0.0667
4	3.5	3.625	3.75	0.17301	0.06910	-0.03440	0.01196	0.03448

After four iterations, the absolute relative error falls below 5%, yielding an estimated root of 3.625. Inserting $c = 3.625$ into our equation for velocity yields 35.0691 m/s, an error of 0.2% from the inputted velocity of 35 m/s. $\left(\left| \frac{35.0691-35}{35} \right| \times 100\% \approx 0.1974\% \right)$

```

% Matlab initialization

close all; % closes all plots

clear; % clears memory

clc; % clears command window


% set model parameters

g = 9.81;

t = 3.9;

m = 80;

v = 35;


% Set initial variables

xLow = 3;

xUpp = 5;

errorTol = 0.05;

iterMax = 10;


% Set up a function handle for convenience

dragFun = @(c) (g*m/c)*(1 - exp(-c/m*t))-v;


% Start the bracketing method. This implimentation does take some steps to
% minimize the number of function evaluations for efficiency. Specifically,
% it save values from previous iterations so only one new evaluation is
% needed for each new iteration.

error = 1;

iter = 1;

% Compute functions at bounds

fLow = dragFun(xLow);

```

```
fUpp = dragFun(xUpp);
```

```
% Pretty-printing
```

```
fprintf(' iter| xLow| xMid| xUpp| f(xLow)| f(xMid)| f(xUpp)| fBracket| Error\n')
```

```
% Iterate until our error falls below tolerance or we reach the maximum
```

```
% number of iterations. The latter option prevents the computer from
```

```
% falling into an infinite loop. NOTE: hitting ctrl+c will cause Matlab to
```

```
% force-quit its current process, breaking infinite loops.
```

```
while (error > errorTol) && (iter <= iterMax)
```

```
    % Compute the midpoint and function at midpoint
```

```
    xMid = 0.5*(xLow + xUpp);
```

```
    fMid = dragFun(xMid);
```

```
    % Compute fLow*fMid
```

```
    fBracket = fLow*fMid;
```

```
% This code pretty-prints data
```

```
fprintf('%4d | %2.4f| %2.4f| %2.4f| %10f| %10f| %10f| %10f|',...
```

```
    iter, xLow, xMid, xUpp, fLow, fMid, fUpp, fBracket)
```

```
% Start logic for bracket replacement
```

```
if (fBracket < 0) % replace xLow with xMid
```

```
    xUpp = xMid;
```

```
    fUpp = fMid; % Minimizes function evaluations
```

```
    error = abs((xMid-xLow)/xMid);
```

```
else % replace xHigh with xMid
```

```
    xLow = xMid;
```

```
    fLow = fMid; % minimizes function evaluations
```

```
    error = abs((xMid-xUpp)/xMid);  
end  
  
% More pretty-printing  
fprintf(' %2.5f\n', error)  
  
% increase iteration count  
iter = iter + 1;  
end
```